SAE S2.04 - Livrable 1

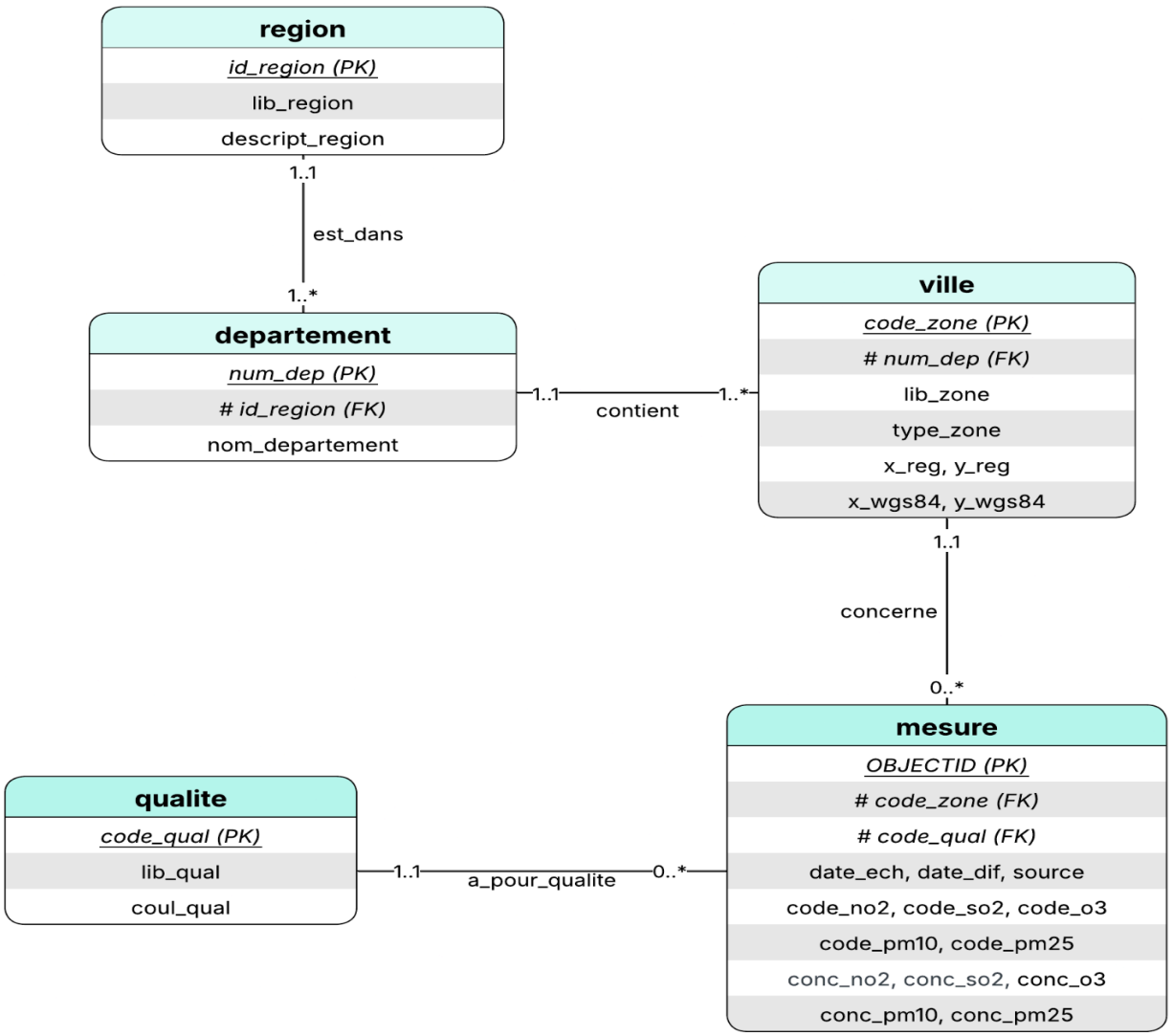
Diagramme UML, Modèle Physique de Données et argumentation

Note : Le document fait 5 pages !

*Avant d'exécuter les scripts de création de table (**script de création des tables.sql**) et la migration de données (**script de migration des données.sql**), IL faut s'assurer que la base de données d'origine (**create table qualite_air.sql**) existe déjà sur Pedaweb. Car on travaille sur la base de données fournie.*

1. **Diagramme UML**

DIAGRAMME UML



2. Modèle Physique de Données

Table region

Colonne	Type	Contrainte
id_region	serial	PK
lib_region	varchar	
descript_region	varchar	

Table departement

Colonne	Type	Contrainte
num_dep	varchar	PK
id_region	int	FK -> region(id_region)
nom_departement	varchar	

Table qualite (nouvelle)

Colonne	Type	Contrainte
code_qual	int	PK
lib_qual	varchar	
coul_qual	varchar	

Table ville (nouvelle)

Colonne	Type	Contrainte
code_zone	varchar	PK
lib_zone	varchar	
type_zone	varchar	
x_wgs84	real	
y_wgs84	real	
x_reg	real	
y_reg	real	
num_dep	varchar	FK -> departement(num_dep)

Table mesure (nouvelle)

Colonne	Type	Contrainte
OBJECTID	int	PK
date_ech	date	
date_dif	date	
source	varchar	
code_zone	int	FK -> ville(code_zone)
code_qual	int	FK -> qualite(code_qual)
code_no2	int	
code_so2	int	
code_o3	int	
code_pm10	int	
code_pm25	int	
conc_no2	real	
conc_so2	real	
conc_o3	real	
conc_pm10	real	
conc_pm25	real	

3. Argumentation des choix de conception

3.1. Pourquoi découper la table qualite_air en plusieurs tables ?

La table d'origine qualite_air contient 25 colonnes qui mélangent des informations de natures différentes : des infos sur la ville (code, libelle, coordonnées GPS), des infos sur la mesure (date, source), les indices de qualité et les concentrations des polluants. Résultat : les mêmes informations sont répétées à chaque ligne. Par exemple, le libellé "Lyon" et ses coordonnées GPS sont répétés à chaque nouvelle mesure de la ville.

Pour éviter cette redondance, on découpe la table en plusieurs tables, chacune correspondant à une entité bien identifiée :

- Une table pour les villes
- Une table pour les niveaux de qualités de l'air (référentiel)
- Une table pour les mesures.

3.2. Pourquoi une table qualité séparée ?

Dans la table d'origine, les colonnes code_qual, lib_qual et coul_qual vont toujours par trois. Par exemple, code_qual = 1 est toujours associé à lib_qual = 'Bon' et coul_qual = '#50F0E6'. Comme il n'y a que **6 valeurs possibles** (de 1 à 6), ça n'a aucun intérêt de répéter "Bon" ou "#50F0E6" des milliers de fois.

On crée donc une table qualite avec seulement 6 lignes, et dans la table mesure on ne garde que code_qual (en clé étrangère) qui pointe vers la bonne ligne de qualite. Ça évite la redondance et ça évite aussi les erreurs de saisie (imaginons quelqu'un qui écrirait "bonne" au lieu de "Bon").

3.3. Pourquoi une table ville séparée ?

Toutes les colonnes qui décrivent une ville (code_zone, lib_zone, type_zone, coordonnées GPS) sont **indépendantes de la date de la mesure**. Lyon a toujours les mêmes coordonnées, que la mesure soit faite le 1er janvier ou le 15 août. On créé donc une table ville qui contient une ligne unique par ville, et la table mesure référence la ville via code_zone en FK.

3.4. Pourquoi garder toutes les colonnes de polluants dans mesure ?

On aurait pu imaginer faire encore plus de tables (une pour les sous-indices, une pour les concentrations, une par polluant...). Mais les 10 colonnes (5 sous-indices + 5 concentrations) décrivent toutes le même évènement : un relevé pour une ville à une date donnée. Les sortir dans d'autres tables aurait obligé à faire des jointures supplémentaires pour chaque requête, sans apporter de bénéfice particulier (il y a toujours exactement 5 polluants).

On choisit donc de garder les 10 colonnes dans mesure pour simplifier les requêtes de la Partie 2.

3.5. Pourquoi un lien ville -> departement -> region ?

L'énoncé demande que la base puisse gérer **toutes les régions de France**, pas uniquement Auvergne-Rhone-Alpes. Pour cela, on s'appuie sur les tables region et departement déjà fournies. Le lien se fait naturellement par le num_dep dans la table ville : il suffit de le déduire du code_zone (les 2 premiers chiffres du code INSEE correspondent au numéro du département, à l'exception des départements à un chiffre).

De cette manière, on peut, pour n'importe quelle mesure, remonter : mesure -> ville -> departement -> region.

3.6. Justification des types choisis

Quelques choix de types méritent d'être expliqués :

- **num_dep** en varchar et pas en int : parce que certains départements ont des codes comme "2A" ou "2B" (Corse), ou "971 à 976" (outre-mer). Un type entier ne pourrait pas les stocker.
- **code_zone** en varchar : contrairement à num_dep, les codes INSEE des communes sont toujours numériques.
- **date_ech** en date (et pas en varchar) : pour pouvoir faire des comparaisons et des tris chronologiques si besoin.
- Les concentrations en float : parce que ce sont des mesures physiques avec des décimales (ex : 42.6 µg/m³).
- Les codes d'indice en int : valeurs entières de 1 à 6.

3.7. Le choix des clés primaires

Chaque table a une clé primaire naturelle :

- **region** : id_region (déjà donnée)
- **departement** : num_dep (déjà donnée)
- **qualite** : code_qual (de 1 à 6, unique par niveau)
- **ville** : code_zone (le code INSEE est unique pour chaque commune)
- **mesure** : OBJECTID (déjà fourni dans le CSV, unique pour chaque mesure)

On n'a donc pas eu besoin d'inventer de nouveaux identifiants : les données d'origine en fournissaient déjà.

Conclusion : Avec mon binôme Toure Issahou Djawad, nous avons choisi de faire apparaître directement les clés primaires et étrangères sur le diagramme UML. Cela permet de visualiser plus facilement quelle clé "migre" vers quelle table à travers les cardinalités des relations.